Ders 1 Genişletilmiş Notu: Yazılım Mühendisliğine Giriş: Yazılım, Yazılım Türleri, Yanılgılar ve Temel Süreç Adımları

Ders 1 Genişletilmiş Notu: Yazılım Mühendisliğine Giriş: Yazılım, Yazılım Türleri, Yanılgılar ve Temel Süreç Adımları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Yeri, Kaynakları ve Kapsamı
- 2. Yazılım Nedir?
- 3. Bakım ve Yazılım Yaşam Döngüsü
- 4. Yazılım Türleri
- 5. Eski Yazılım (Legacy Software)
- 6. Yazılım Hakkındaki Yanılgılar
- 7. Mühendislik Nedir?
- 8. Yazılım Mühendisliğinin Tanımı
- 9. CASE Araçları
- 10. Yazılım Geliştirme Sürecinin Temel Adımları
- 11. Destek (Şemsiye) Aktiviteleri ve Yazılım Konfigürasyon Yönetimi
- Örnek Sorular
- Dersin Toparlaması

1. Dersin Yeri, Kaynakları ve Kapsamı

1.1 Sistem Analizi Dersinden Yazılım Mühendisliğine

Hoca derse, öğrencilerin bir önceki yıl Sistem Analizi ve Tasarımı dersini aldığını hatırlatarak başladı. O derste **bilgi sistemi geliştirme süreci** görülmüştü. Bu derste aynı adımlar, bu kez yalnızca yazılım için, daha ayrıntılı ve nesneye yönelik (object-oriented) kavramlar üzerinden işlenecek. Amaç, geliştirme sürecindeki adımların modellemesini, analizini ve tasarımını oluşturmaktır.

Sistem analizinde “sistem analizi” ve “sistem tasarımı” başlıkları altında, uygulamanın donanım ve altyapısını kurmak için yapılan işler de ele alınmıştı. Bu derste ise yalnızca yazılım geliştirirken yapılanlar önemlidir. Örneğin gereksinimler, işlevsel gereksinimler (functional requirements) ve işlevsel olmayan gereksinimler (non-functional requirements) olarak ayrıntılı görülecek.

1.2 Kaynak Kitaplar

Hoca üç kaynak kitap tanıttı:

- **Roger Pressman, Software Engineering:** Hocanın ve dersi veren diğer hocanın kullandığı temel kitap. Dersin ilk konuları (yazılım nedir, yazılım türleri, süreç modelleri, gereksinim mühendisliği) bu kitaptan işlenecek.

- **Ian Sommerville, Software Engineering:** İkinci temel kitap. Bir grup hoca bu kitabı Türkçeye çevirmiş.
- **Craig Larman'ın UML ve kalıplar kitabı:** Nesneye yönelik analiz, tasarım ve modelleme aşaması için kullanılacak. Tasarım kalıpları (design patterns) da bu kitaptan anlatılacak.

Hocaya göre Pressman ve Sommerville'in kitapları dünyada en çok kullanılan yazılım mühendisliği ders kitaplarıdır ve üniversitelerde aynı derecede kullanılır. İkisi de çok kapsamlıdır. Bu yüzden derste bütün konulardan bahsedilecek, ama hepsinin ayrıntısı görülmeyecek. Ayrıntıların bir kısmı seçmeli derslerde, bir kısmı sektördeki eğitimlerde karşınıza çıkacak.

1.3 Yazılım Mühendisliği Kitaplarının Dört Ana Kısmı

Hoca, bu kitaplarda aynı adlarla olmasa da genelde dört temel kısım bulunduğunu söyledi:

1. **Yazılım mühendisliğine giriş ve süreç modelleri:** Süreç modellerinin kullanımı, seçimi, ayrıntılı tanımı ve yazılım mühendisliğinin temel adımlarının anlaşılması.
2. **Yazılım mimarisi (software architecture):** Yazılım tasarımının profesyonel olarak nasıl yapılacağı. Bölümde lisansüstünde ayrı bir ders olarak okutuluyor.
3. **Yazılım proje yönetimi:** Büyük yazılım projelerinin nasıl takip edileceği, zaman ve kaynak planının nasıl çıkarılacağı, risk yönetiminin nasıl yapılacağı. Bazı kitaplarda "yazılım yönetimi" diye geçer. Lisansüstünde ayrı bir ders olarak var.
4. **Kalite yönetimi ve test teknikleri:** Yazılım testi (sınama) temel süreç adımlarından biridir, ama kalite yönetimi kapsamında daha ayrıntılı ele alınır. Kalite için oluşturulan sertifika yolları da bu kısımdadır.

Hoca dördüncü kısımla ilgili dersin lisansüstünde yaklaşık 20 yıldır okutulduğunu, lisansa da seçmeli olarak yaklaşık 10 yıl önce eklendiğini anlattı. Lisansa eklenmesinin nedeni, sektörde test ve kalite ekiplerinin artması ve bu alanda çalışacak mezunların aranmasıdır.

1.4 Dersin İçeriği

Derste işlenecek konular şunlardır:

- Yazılım kavramı, yazılım mühendisliği kavramı ve yazılım türleri (bu haftanın konusu).
- Yazılım geliştirme süreç modelleri (bu hafta başlangıç, gelecek hafta ayrıntı).
- Gereksinim mühendisliği ya da gereksinim analizi.
- Nesneye yönelik çözümleme ve nesneye yönelik tasarım.
- Yazılım test (sınama) teknikleri.
- Yazılım proje yönetimine giriş. Gantt diyagramları sistem analizinde kısaca görülmüştü.
- Yazılım ölçütleri (software metrics).
- Yazılım konfigürasyon yönetimi ve yazılım risk yönetimi. Hoca bunların proje yönetimi ve kalite yönetimi içinde de sayılabileceğini söyledi.
- Tasarım sırasında biraz yazılım mimarisi ve Larman'ın kitabından tasarım kalıpları.

Sistem analizi dersinde temel adımlar dışındaki **destek aktiviteleri** üzerinde hiç durulmamıştı. Bu derste onlar da ayrıntılı görülecek.

Hoca, nesneye yönelik programlama dersinde hangi diyagramların görüldüğünü sordu. Kendi cevabına göre o derste use case yapısı üzerinde çok durulmuyor, ama ardışıl diyagramlar (sequence diagrams) ve sınıf diyagramları (class diagrams) çiziliyor. Bu derste bunların hepsi kullanılarak nesneye yönelik çözümleme ve tasarımın projede uygulanması isteniyor. Nesneye yönelik programlama dersini almamış olanlar Larman'ın kitabından daha fazla yararlanmalıdır.

1.5 Yazılım Ölçütleri Üzerine İlk Not

Hoca "ölçüt" kelimesini "metrik" anlamında kullandığını belirtti. "Software metrics" adıyla pek çok kitap bulunur. Soru şudur: Bir yazılım geliştirdik ya da bir yazılım satın alınacak; bu yazılımı neye göre ölçeceğiz?

- Yazılımın iç özellikleri ve dış özellikleri.
- Proje geliştirirken ölçülebilecek şeyler.

- Geliştirilen yazılımın maliyetini ve satış fiyatını belirlerken kullanılacak ölçütler.
- Bir yazılım projesine başlarken maliyet tahmini için gereken ölçütler: büyüklük ölçütleri, karmaşıklık ölçütleri, algoritma yapısı ölçütleri.

Hocanın benzetmesi: Bir şey aldığınızda ağırlığını kilogramla, uzunluğunu metreyle ölçersiniz. Peki yazılımın kalitesini, karmaşıklığını ya da uzunluğunu ölçerken hangi birimi kullanacağız? Bu soruların cevabı dersin içeriğindedir.

2. Yazılım Nedir?

Hoca, üçüncü sınıfa gelen öğrencilerin donanım ve yazılım kavramlarını bildiğini, ama konuyu daha ayrıntılı ele alacaklarını söyledi. Tanım şöyledir:

> Herhangi bir boyuttaki, herhangi bir donanımda çalışan bilgisayar programını ve basılı ya da elektronik ortamdaki belgelerin hepsini içeren ürüne **yazılım** denir.
>
> **Hocanın vurgusu:** Yazılım eşittir program değildir. Bu derste "yazılım = program" yanlışını kesinlikle yapmamanız gerekiyor.

Bunun nedeni şudur: Artık bir programı tek kişi geliştirmiyor. Program bir ekip çalışmasıyla geliştiriliyor ve daha sonra bakımını, yani düzeltme ve değişiklik işlerini belki başkaları yapıyor. Bu yüzden yazılım üzerinde daha sonra çalışacak olanların takip edebilmesi için belge hazırlanması gerekir. Programla ilgili belgeler de yazılımın bir elemanıdır.

Ayrıca programda kullanılan veri tanımları, veri yapıları, veri tabanı, veri saklama ortamları ve bunlarla ilişkiler de yazılımın içindedir. Dersin sonunda hoca bu tanımı test verileriyle (test case'lerle) tamamladı (bkz. Bölüm 11). Böylece yazılımın elemanları şöyle özetlenebilir:

- Program.
- Basılı ya da elektronik belgeler.
- Veri tanımları, veri yapıları ve veri tabanı.
- Test verileri (test case'ler).

3. Bakım ve Yazılım Yaşam Döngüsü

3.1 Yazılım Eskimez, Ama Yetersiz Kalır

Yazılım, "software" kelimesinin de anlattığı gibi fiziksel değil, "soft" bir üründür. Bu yüzden bir makine parçası gibi aşınıp eskimesi söz konusu değildir. Önemli olan yazılımın **yetersiz kalmasıdır**. Örneğin bir yazılım, işletim sisteminin ya da veri tabanı yazılımının yeni bir sürümü altında yetersiz kalabilir. Değişim kaçınılmazdır ve yazılım üzerinde değişiklikler yapılır.

3.2 Bakım (Maintenance)

Yazılım üzerinde yapılan bu değişikliklere **bakım (maintenance)** denir. Hoca bakımın farklı nedenlerini saydı:

- **Hata düzeltme:** Bir hata oluşur ve onu düzeltmek için değişiklik yapılır.
- **Ortamdaki yenilikler:** Hata olmasa bile yazılımın geliştirildiği programlama dilinde ya da veri tabanında yenilikler olur. Bu yenilikleri yazılıma uyarlamak için değişiklik gerekir.
- **Uygulama alanındaki değişiklikler:** Müşteriden gelen değişiklik istekleri. Hata yoktur, ama uygulamanın yapısı değişmiştir ve yazılımın da bu değişikliğe uyarlanması istenir.

Bir süre sonra ne kadar bakım yapılırsa yapılsın yazılımın kullanılmaz hale geldiği görülebilir. Bu durumda, ne kadar ekleme ve değişiklik yapılmış olursa olsun, yazılımın kullanımdan tamamen kaldırılması ve yenilenmesi gerekir.

3.3 Yazılım Yaşam Döngüsü (Software Life Cycle)

> **Yazılım yaşam döngüsü:** Yazılımın bir fikir olarak doğmasından geliştirilmesine, kullanılmasına, kullanım süresinin bakımla sürdürülmesine, sonra bir süre desteksiz kullanılmasına ve sonunda kullanım dışı bırakılmasına kadar geçen süreçtir.

Aşamalar sırayla şöyle özetlenebilir:

1. Fikrin doğması.
2. Geliştirme.
3. Bakım yapılarak kullanım.
4. Desteksiz kullanım.
5. Kullanım dışı bırakılma.

Dersin hedefi yaşam döngüsündeki bütün aşamaları görmektir. Hoca, bir bilgisayar mühendisinin bu aşamaların her birinde görev alabileceğini vurguladı:

- Yeni bir proje fikri yaratan ekipte.
- Projenin başlangıcında, fizibilite analizi ve gereksinim analizinde.
- Tasarımda, yazılım mimarı (software architect) olarak.
- Kod yazan ekipte.
- Test ekibinde (tester olarak).
- Kullanılan yazılımların bakımını yapan ekipte.

Bu yüzden her aşamanın bilinmesi gerekir.

4. Yazılım Türleri

Hoca yazılım türlerini, diğer derslerde de görüldüğü için çok genel olarak sınıfladı:

Tür	Açıklama ve örnekler
Sistem yazılımları	İşletim sistemleri dersinde görülen türler: işletim sistemleri, derleyiciler
Mühendislik ve bilimsel yazılımlar	Büyük hacimli verilerle uğraşan uygulamalar; veri madenciliğindeki büyük veri uygulamaları, istatistiksel analiz
Gömülü (embedded) yazılımlar	Donanımla ilişkili, denetim amaçlı, gerçek zamanlı uygulamalar
Uygulama yazılımları	Bir şirkette kullanılan yazılımlar: muhasebe yazılımları, otomasyon programları, Word ve Excel gibi araçlar
Hazır paketler (COTS)	Satışa hazır yazılımlar; örneğin bir hastane yazılım paketi
Ürün hatları (product line)	Bir yazılım evinin bir alanda sürekli geliştirdiği ürün ailesi; yazılım kütüphaneleri
Kurumsal yazılımlar	Bir şirkette ticari bir iş için kullanılan yazılımlar; ERP paketleri

4.1 Hazır Paketler (COTS)

COTS, Commercial Off-The-Shelf, yani satışa hazır, rafta duran ticari yazılım demektir. Hocanın örneği bir hastane yazılım paketidir.

4.2 Ürün Hatları (Product Line)

Ürün hattında bir yazılım evi, bir alanda geliştirdiği ürünü sürekli geliştirir. Bunun için o alana özgü bir alan analizi (domain analysis) yapar, o alana özgü bir dil oluşturur ve hazırladığı bir platform üzerinde ürünlerini sürekli geliştirir. Hocaya göre bu, fabrikalardaki üretim hattı fikrinin yazılıma uyarlanmasıdır ve yazılım evlerinde böyle bir yöntem vardır. Bu kavram ilerleyen derslerde tekrar karşımıza çıkacak.

4.3 Yeniden Kullanılabilirlik (Reusability)

Hoca, ürün hatlarının ve COTS'un aslında yazılımı daha hızlı ve belli standartlarda üretmek için olduğunu söyledi. İkisi de **yeniden kullanılabilirliği (reusability)** sağlar.

> **Yeniden kullanılabilirlik:** Yazılım geliştirme konusunda yapılan bir çalışmanın, tekrar emek harcamadan başka işlerde de kullanılabilmesidir.

Yeniden kullanılabilirlik önemli kalite özelliklerinden biridir. Ürün hatları, COTS ve tasarım kalıplarının hepsi bu konuda fayda sağlar.

4.4 Kurumsal Yazılımlar

Kurumsal yazılımlar bir şirkette ticari bir iş için kullanılan yazılımlardır. Genellikle ya müşteriye özel tasarlanırlar ya da ERP paketleri gibi hazır paketler o kuruma özgü olarak uyarlanır.

5. Eski Yazılım (Legacy Software)

Aradan sonra hoca eski yazılım kavramına geçti. Kurumsal yazılımlar genelde bir şirketin iş sürecinin parçasıdır. Bunların bir kısmı çok uzun süredir kullanılır ve önemli işleri halleder. Örnekler: muhasebe yazılımı, stok kontrol yazılımı, maaş işlerinin hazırlandığı bordro yazılımı. Bu yazılımlara kimse dokunmak istemez, herkes dokunmaya korkar.

Eski yazılımların özellikleri şunlardır:

- Eksik ya da hatalı belgeleri vardır.
- Zamanla eklemeler ve yamalarla karmaşıklaşmış bir kodu vardır.
- Esnek olmayan bir yapısı vardır.
- Donanımla daha fazla ilişkilidir.
- Yazılım mühendisliğindeki gelişmelerden yoksun oldukları için daha düşük kalitededir.

İş alanındaki yeni gereksinimler nedeniyle ve güncel sistemlerle birlikte çalışabilmeleri için bu yazılımlar değiştirilmek istenir. Ama bakımları zor olduğu için zorlukla bakım yapılarak bir süre daha kullanılırlar. Bu tür yazılımlara **eski yazılım (legacy software)** denir.

Hocanın örneği COBOL'dur. Çok eskiden COBOL dilinde yazılmış programlar vardı ve 2000 yılına geçişte Avrupa'da bu yazılımlarda güncelleme yapacak COBOL programcıları aranmıştı.

> **Ek bilgi (derste anlatılmadı):** 2000 yılına geçiş sorunu "Y2K" ya da "milenyum hatası" olarak bilinir. Eski programların çoğu yılı iki haneyle sakladığı için 2000 yılının 1900 ile karışması büyük olasılıkla bu sorunun temel nedeniydi. Bu yüzden eski COBOL kodlarının taranıp düzeltilmesi gerekmişti.

6. Yazılım Hakkındaki Yanılgılar

Hoca, bir kurumda yazılım projesi geliştirilirken temelde üç taraf bulunduğunu söyledi:

- **Müşteri tarafı:** Büyük bir şirketin yazılım birimi için diğer departmanlar (mali işler, satış vb.) müşteri departmanıdır. Bir yazılım firması için ise yazılımı satacağı taraf müşteridir.
- **Yönetim tarafı:** Yazılım firmasının ya da genel amaçlı bir şirketin yöneticisi.
- **Programcı tarafı:** Yazılımı geliştirenler.

Her tarafın yazılım hakkında yanılgıları (software myths) vardır.

6.1 Müşterinin Yanılgıları

Yanılgı 1: "Programın yazılmasına başlamak için amaçları genel olarak belirlemek yeter. Ayrıntılar daha sonra, ben istediğim zaman karşılanabilir. Nasıl olsa yazılım esnektir."

Bu her zaman mümkün değildir. Müşteri önce genel olarak ne istediğini anlatmalı, sonra ayrıntıları tek tek belirtmelidir. Gereksinimler başta belirtilmezse temel doğru atılmamış olur.

Yanılgı 2: "Yazılım esnektir. Değişen gereksinimler sisteme kolayca uyarlanabilir." Yani gereksinim analizi bittikten sonra da yeni gereksinim eklenebilir ya da yanlış belirtilen gereksinimler düzeltilebilir.

Burada gereksinimden kastedilen, **işin nasıl yapıldığının anlatılmasıdır**. Bir müşteriye yazılım hazırlanırken müşterinin yaptığı işi anlatması gerekir. Önce iş süreci bilinmelidir ki geliştirilecek yazılım doğru analiz edilebilsin.

> **Hocanın vurgusu:** Yaşam döngüsü ilerledikçe değişen gereksinimleri yazılıma uyarlamanın bedeli üstel olarak artar.

Hoca bu yanılgıları maddeler halinde verdiklerini, ama özünde hepsinin aynı olduğunu söyledi: Müşterinin, yazılımcıların proje planında, gereksinim analizinde ve tasarımda çok kolay değişiklik yapabileceğini sanması. "Yazılımcıların yaptığı her şey esnektir, sürekli ekleme ve değişiklik yapabiliriz" kanısı yazılım geliştirmeyi kötü yönde etkiler. Yazılımcıların da gereksinim analizi sırasında müşteriden olabildiğince doğru ve yeterli bilgiyi almaya çalışması ve bunun için gerekli kontrolleri yapması gerekir.

6.2 Yönetimin Yanılgıları

Yanılgı 1: "İş yetişmiyorsa takıma yeni programcılar ekleriz."

Bu bir yanılgıdır. Yetişmeyen bir iş için yazılım proje ekibine sonradan eleman eklemek doğru bir yaklaşım değildir ve çözüm sağlamaz. Hocaya göre bu, işi üstel olarak zorlaştıran ve daha çok geciktiren bir yaklaşımdır.

> **Ek bilgi (derste anlatılmadı):** Bu ilke literatürde **Brooks yasası** olarak bilinir: "Geciken bir yazılım projesine insan eklemek onu daha da geciktirir." Fred Brooks bunu büyük olasılıkla 1975 tarihli *The Mythical Man-Month* kitabında dile getirmiştir. Temel neden, yeni gelenlerin işi öğrenmesinin zaman alması ve kişi sayısı arttıkça iletişim yükünün hızla büyümesidir.

Yanılgı 2: "Yazılımı tamamen ya da kısmen fason yaptırırım."

Girişimci olan ama yazılımcı olmayan bir yönetici bir yazılımı geliştirmeyi üstlenir ve gerekirse işi dışarıya yaptıracağını düşünür. Elbette ek iş gücü ya da dışarıdan bir taşeron firma çalıştırılabilir, ama bu **baştan planlanmalıdır**. Hocanın örneği: Bir yazılım geliştirilirken ekran tasarımları bir grafik tasarım şirketine yaptırılabilir, ama bu baştan planlanır. Bir programlama işini alıp sonradan "şu yarısını da şunlara yaptırayım" diye düşünmek proje yönetimi fikrine aykırıdır.

Buna bağlı bir ilke daha vardır: Proje ilerlemesini kendi içinde denetleyemeyen bir firma, dışarıya verdiği işi izlemekte de zorlanacaktır. Yönetim yazılım projelerinde de ilerlemeyi izlemelidir. Dışarıya verilecek iş, denetiminin nasıl yapılacağı bilinerek baştan planlanmalıdır. "Son dakikada yetişmedi, bunu da başka firmaya yaptırayım" diye düşünmek hatalıdır.

Yanılgı 3: "Açık kaynaklı yazılım üretirsem kâr edemem."

Açık kaynaklı yazılımlarda kâr danışmanlık hizmetlerinden gelir ve bunlar için iş modelleri geliştirilmektedir. Hoca, son yıllarda özellikle açık kaynaklı yazılımların teşvik edildiğini söyledi.

6.3 Programcının Yanılgıları

Yanılgı 1: "Yazılımı hazırladım, bitirdim, teslim ettim; benim işim bitti."

Böyle bir şey yoktur. Yazılımı geliştirmek için harcanan çabanın, iş gücünün hemen hemen bir o kadarı müşteriye teslimden sonra da harcanır. En azından kullanıcı alışana kadar bir süre destek vermek gerekir.

Hoca kendi deneyiminden örnek verdi: Üniversite içinde hazırladıkları yazılımlarda, kullanıcılar kullanmaya başladıktan sonra "hata var mı?" diye sormamak lazım, çünkü müşteri hemen "hata var" der. Ama arada bir uğrayıp bakmak gerekir. Hocanın eski bir hocası "arada kullanıcılara çay kahve içmeye gitmek gerekir" dermiş. Yani bitirdiğiniz yazılımı takip etmeniz gerekir.

Yanılgı 2: "Yazılımı tamamlamadan kalitesini ölçemem."

Bu da yanlıştır. Kalite yönetimi yazılım mühendisliğinin önemli konularından biridir ve her aşamada kalite yönetimiyle ilgili yapılacak işlemler vardır. Örneğin analiz sırasında modeller oluşturulur ve çözümlemede ne yapıldığı **SRS raporunda** (Software Requirements Specification, yazılım gereksinim belirtimi) belirtilir. Bu da ölçülmelidir: SRS raporu doğru mu? Gereksinimler doğru belirlendi mi? Her aşamanın kalite ölçütleri vardır.

Yanılgı 3: "Yazılım eşittir program."

Hoca bunu daha önce de vurguladığını hatırlattı. Yazılım geliştirme, gereksinim analiziyle başlar ve şunları içerir:

- SRS raporu, tasarım raporu gibi belgeler.
- Test çalışmaları.
- Entegrasyon çalışmaları: diğer yazılımlarla uygun girdi ve çıktı alışverişi.
- Bakım belgesi ve kullanıcı belgesi.

Bunların hepsi yazılım geliştirmenin içindedir. Yani iş yalnızca program yazmak değil, yazılım geliştirmektir.

Yanılgı 4: "Önemli olan çalışan programı hazırlamaktır. Kodu yazdım, çalışıyor, kullanıcı da çalıştırsın. Çok güzel algoritmalar kullandım; başkaları benim kadar güzel algoritma kullanamaz."

Bu da bir yanılgıdır. Yaptığınız işi çok iyi yapmanız gerekir, ama işin eksiksiz olması da gerekir. Yazılım mühendisliğinin gereklerini uygulayarak geliştirmeli, yaptığınız işi belgelemeli ve başkalarının da anlayacağı hale getirmelisiniz. Böylece daha sonra bakım yapacak programcılar rahat çalışabilir. Bunun için:

- Geliştirme yoluyla ilgili belgeler hazırlanmalı.
- Hangi verilerle test edildiği ve hangi testlerin yapıldığı belirtilmeli.
- Kullanıcılara yazılımı nasıl çalıştıracaklarını anlatan belgeler hazırlanmalı.

Bunların hepsi kalite için harcanan çabadır ve yaşam döngüsündeki diğer aşamaların daha kolay yapılmasını sağlar.

Hoca bu bölümü şöyle bağladı: Bu kadar çaba harcanmasına rağmen yazılım projelerinin çoğu hâlâ başarısız oluyor. Bunun nedeni olarak yazılımların proje yönetimi kapsamında hazırlanması gerekliliği düşünülebilir.

7. Mühendislik Nedir?

Yazılım mühendisliğini tanımlamadan önce hoca, "yazılım" kelimesinin üzerinde durulduğunu, şimdi "mühendislik" kelimesine bakılacağını söyledi ve öğrencilere "Mühendis nedir?" diye sordu.

Soru: Mühendis nedir? **Cevap (bir öğrenci):** Bir sorunla karşılaştığında sorunun çözümünü sağlayacak ve bunun için bir sistem geliştirebilecek kişi.

Hoca bu tanımı kabul edip tamamladı: Mühendis bir problemle karşılaştığında çözüm arar, ama ilk akla gelen çözümü hemen uygulamaz. Uygulamaya geçmeden önce mühendislikte iki temel şey vardır.

7.1 Tasarım

Bir işe hemen başlamak değil, önce tasarımını yapmak gerekir. Yazılım geliştirme süreçlerinde de "yazılım mühendisliği" kavramını oturttuğumuz için hemen program yazmaya başlamayız: önce analiz, sonra tasarım, sonra geliştirme.

> **Hocanın vurgusu:** Hemen kod yazmaya başlanmaz. Önce analiz yapılır, sonra tasarım, sonra geliştirme. Bu süreçte tasarım çok önemlidir.

Hoca, Türkiye'de mühendislik bölümlerinin akreditasyonunu yapan MÜDEK'in ve uluslararası ABET akreditasyonunun da programlarda tasarımla ilgili ne kadar çalışma yapıldığına baktığını söyledi. Bilgisayar mühendisliğinde bu, yazılıma başlamadan önce tasarım modellerinin oluşturulmasıyla sağlanır.

7.2 Ölçme

"Mühendis" kelimesinin kökünde **hendese** kavramı vardır. Mühendis, hendese yapan, yani ölçmeyi bilen kişidir. Hoca bunu "mühendis ölçen, biçen kişidir" diye özetledi. Yazılımda bu şu sorulara karşılık gelir:

- Geliştirdiğimiz bir yazılımın ne büyüklükte olduğunu ölçebiliyor muyuz?
- Ne kadar karmaşık olduğunu ölçebiliyor muyuz?
- Geliştireceğimiz bir yazılımın, yapacağımız bir projenin ne büyüklükte olacağını ölçebiliyor muyuz?

> **Hocanın vurgusu:** Mühendislik kavramında iki anahtar kelime vardır: **ölçme** ve **tasarım**. Geliştirdiğiniz yazılımı ne kadar ve hangi yollarla ölçebiliyorsunuz, yapacağınız işi nasıl tasarlayabiliyorsunuz?

8. Yazılım Mühendisliğinin Tanımı

Yazılımın ve mühendisliğin tanımı yapıldıktan sonra hoca yazılım mühendisliğini tanımladı. Hendese ölçü bilimidir. Bu yüzden yazılım mühendisliğinin özü, geliştirilen yazılıma ölçüm getirmek ve sistematik, disipline edilmiş, ölçülebilir bir yaklaşım uygulamaktır.

8.1 Klasik Tanım ve Üç Anahtar Kelime

Hoca, yazılım mühendisliği kavramının ilk olarak 70'li yıllarda Boehm tarafından ortaya atıldığını söyledi.

> **Ders sonrası not:** Hoca kavramın 70'lerde Boehm tarafından ortaya atıldığını söyledi. "Software engineering" terimi büyük olasılıkla 1968'deki NATO Yazılım Mühendisliği Konferansı ile yaygınlaştı. Hocanın aktardığı "kaliteli yazılımı ekonomik yolla elde etmek üzere mühendislik prensiplerinin kullanılması" tanımı da büyük olasılıkla bu konferansın düzenleyicilerinden Fritz Bauer'e aittir. Barry Boehm'in 1976 tarihli "Software Engineering" makalesi ise alanın önemli erken çalışmalarından biri olarak bilinir.

Hoca tanımın özgün halinde "bilgisayarda etkin çalışan yazılım" ifadesinin geçtiğini, tanımda üç anahtar kelime bulunduğunu söyledi:

1. **Kaliteli yazılım:** Yazılımı ilk akla gelen şekilde değil, en iyi yaklaşımı düşünerek ve en iyi algoritmayı bularak geliştirmek; ayrıntılı belgelerini hazırlamak, daha doğrusu bakım yapacak herkesin anlayacağı şekilde belgelemek. Günümüzde bu, yazılımı herhangi bir yolla değil, yazılım geliştirme süreç modellerinden birini benimseyip onun standartlarıyla geliştirmek demektir.
2. **En ekonomik yol:** Proje yönetimi uygulayarak, zaman planlaması ve kaynak planlaması yaparak yazılımı elde etmek.
3. **Mühendislik prensipleri:** Az önce anlatılan ölçme ve tasarım. Rastgele bir ürün üretmek değil, ürünü ölçülebilir özelliklerde ve kaliteli olarak, tasarım yaparak gerçekleştirmek.

Bu üç anahtar kelimeyle tanım şöyle kurulur:

> **Yazılım mühendisliği:** Kaliteli yazılımı en ekonomik yolla elde etmek üzere mühendislik prensiplerinin kullanılmasıdır.

8.2 Ekonomik ile Ucuz Arasındaki Fark

Hoca, "en ekonomik" ifadesinin "en ucuz" anlamına gelmediğini vurguladı. Ekonomik fizibilite kavramı sistem analizi dersinde görülmüştü.

> **Hocanın vurgusu:** Ekonomik, ucuz demek değildir. Ekonomik kavramı, maliyete karşılık en fazla faydayı elde etmektir. Yalnızca en düşük maliyetli çözüm modelini seçmek ekonomik olmayabilir.

8.3 IEEE Tanımı

Hoca, IEEE'nin 1993 tarihli tanımını da verdi:

> **IEEE tanımı:** Yazılım mühendisliği, yazılımın geliştirilmesine, işletilmesine ve bakımına, yani yazılım yaşam döngüsüne, sistematik, disipline edilmiş ve ölçülebilir bir yaklaşımın uygulanmasıdır.
>
> **Ders sonrası not:** Bu tanım büyük olasılıkla IEEE Std 610.12-1990 yazılım mühendisliği terimleri sözlüğündendir. Pressman tanımı 1993 tarihli bir IEEE standartlar derlemesinden aktardığı için hocanın "1993" demesi büyük olasılıkla bu kaynaktan gelmektedir.

9. CASE Araçları

Hoca, yazılım geliştirme sürecinde destek olan araçlara geçmeden önce ders belgeleriyle ilgili bir soruyu cevapladı ve slaytların bir kısmının paylaşılacağını, ama slaytlarda olmayan açıklamalar yaptığı için kayıtların da izlenmesi gerektiğini söyledi. Ayrıca derste kavramların açıklandığını, nesneye yönelik analiz, tasarım ve modellemeyle ilgili örnekleri başka kitaplardan ve internet kaynaklarından bulup üzerinde çalışmak gerektiğini, özellikle bir problem verildiğinde ek kaynak kullanmanın faydalı olacağını ekledi.

9.1 CASE Nedir?

Hoca sınıfa "CASE aracı nedir?" diye sordu ve açılımını tahtaya yazdı: **Computer-Aided Software Engineering** (bilgisayar destekli yazılım mühendisliği).

Soru: CASE aracı nedir? **Cevap (bir öğrenci):** Örneğin tasarım kalıpları oluştururken, yazılım mimarisini kurarken kullanılan UML araçları olabilir.

Hoca cevabı onayladı ve bir benzetmeyle açıkladı: CAD/CAM uygulamasının ne olduğunu herkes bilir; bilgisayar destekli tasarım ve bilgisayar destekli üretim. Bunun gibi, yazılım geliştirme sürecinde yapılan işleri kolaylaştıran, hızlandıran ve rutin işleri daha iyi yapmamızı sağlayan yazılımlar da hazırlanmıştır. Bunlara **CASE araçları** denir.

9.2 CASE Araçlarına Örnekler

Yalnızca modelleme alanından örnek verilirse, CASE araçları şunları yapar:

- UML diyagramlarını standartlarına uygun çizmemize yardımcı olur.
- Analizde oluşturulan use case diyagramına uyan bir ardışıl diyagram çizmemizi sağlar.
- Sınıf diyagramıyla uyumlu bir ardışıl diyagram çizmemizi sağlar.
- Böylece aşama aşama analiz ile tasarım arasında kontrol ve bağlantı kurar.
- ER diyagramını (Entity-Relationship, varlık-ilişki diyagramı) çizer ve ER diyagramı ile veri akış diyagramı (data flow diagram) arasında tutarlılık sağlar.

Bunlar parça parça araçlardır: analizde kullanılan CASE araçları, tasarımda kullanılan CASE araçları. Örneğin nesneye yönelik programlama dersinde kullanılmış olabilecek **Violet** aracı (hoca adının "menekşe" anlamına geldiğini söyledi) nesneye yönelik modellemede, analiz modelinin oluşturulmasında ve tasarımda kullanılabilir. Bu derste de projelerde kullanılması istenecek.

9.3 Bütünleşik CASE Araçları

Yalnızca analiz ve tasarım diyagramlarını çizen araçlar değil, yazılım geliştirmenin her aşamasında birbiriyle uyumlu destek sağlayan **bütünleşik CASE araçları** da vardır. Hocanın örneği **Rational Rose**'dur. Rational uzun süredir IBM'e aittir ve bir yazılım geliştirme platformu olarak tanıtılır. Bu platform, yazılım geliştirmeyi başından sonuna kadar destekleyen yazılımlardan oluşur.

10. Yazılım Geliştirme Sürecinin Temel Adımları

Hoca, bütünleşik CASE aracından yola çıkarak yazılım geliştirme sürecinin temel adımlarını öğrencilere saydırmak istedi.

10.1 Sınıfta Adımların Bulunması

Bir öğrenci sistem analizi dersinden hatırladıklarını anlattı: önce fikir belirlenir, birden fazla fikir varsa elenir. Hoca bunu "önce ön inceleme, sonra fizibilite analizi; fizibilite analizinde alternatif çözüm önerilerinin en uygunu bulunur" diye düzeltti. Öğrenci ardından bir geliştirme aşamasının başladığını ve sürekli düzeltme yapılarak teste kadar gidildiğini söyledi.

Hoca bunun üzerine sordu: Sürekli düzeltme yapılıyor, ama düzeltmeyi çok fazla yapmamak için önce ne yapıyoruz? Öğrenci önce müşteriyle konuşulduğunu, gereksinimlerin analiz edildiğini söyledi. Hoca: "Birinci aşama gereksinim analizi." Öğrenci bunların diyagramlara dö-

küldüğünü ekledi. Hoca bunları analiz diyagramları olarak adlandırdı: veri akış diyagramları ya da nesneye yönelik yaklaşımda use case diyagramları.

Öğrenci daha sonra yazılım aşamasının başladığını söyleyince hoca araya girdi: Yazılıma geçmeden başka bir model daha oluşturuyoruz. Önce müşterinin gereksinimini anlıyoruz, sonra mühendisliğin gereği olan **tasarımı** yapıyoruz. Tasarım atlanmaz, ondan sonra yazılıma geçilir. Örneğin klasik yolla geliştiriliyorsa tasarımda yapı diyagramı (structure chart) oluşturulur.

10.2 Analiz Modeli ile Tasarım Modeli Arasındaki Fark

Soru: Analiz modeli ile tasarım modelinin farkı nedir? **Cevap (bir öğrenci):** Analiz daha çok sorunun ne olduğuna, nasıl ilerleyeceğine dairdir. Tasarım ise çözümün ne olduğu, programlamanın nasıl yapılacağıdır.

Hoca yorumu doğru buldu ve sistem analizi dersindeki tanımı hatırlattı:

> **Hocanın vurgusu:** Analiz "ne?" sorusuna, tasarım "nasıl?" sorusuna cevap verir.

Analiz (ne?): Üzerinde çalıştığımız problem nedir? Hocanın örneği: Bir müşteri sizden stok kontrolü yapan bir program istedi. Bilgisayarcı olarak önce stokun ne olduğunu, stok işleminde ne yapıldığını anlamalısınız. Bu işi kimlerin yaptığını, hangi aşamalardan geçtiğini öğrenmek için müşteriden iş analizini istersiniz. Bu iş analizi üzerinden ne istediklerini anlamaya çalışırsınız:

- Hangi raporları istiyorlar?
- Saklanacak veri nedir, neler kalıcı olarak saklanmalı?

Bunlarla ilgili use case diyagramları, veri akış diyagramları gibi diyagramlar çizersiniz. Sonra işin ne olduğunu anlamış ve müşteriyle anlaşmış olursunuz.

Hoca burada anlaşmanın önemini sistem analizi dersindeki **salıncak hikâyesi** karikatürüyle anlattı: Müşteri basit bir salıncak tarif eder. Analiz yapan başka bir şey anlar, tasarım yapan başka bir tanım yapar, programcı ondan dinlediklerini anlayarak hatalı bir iş yapar ve sonunda istenmeyen bir şey ortaya çıkar. Belgeleme de yapılmamışsa ortada bir boşluk kalır. Bu yüzden analizde önce ne istendiğini anlamak, sonra ne anladığınızı müşteriye söyleyip doğru anlayıp anlamadığınızı doğrulatmak gerekir.

Bu nedenle derste gereksinim analizi çok ayrıntılı işlenecek. Sistem analizinde gereksinim analizi yalnızca bilgi toplama, modelleme ve raporlama olarak üç adımda görülmüştü. Bu derste 6-7 aşama görülecek.

Tasarım (nasıl?): Ne istendiği anlaşıldıktan ve üzerinde anlaşıldıktan sonra işin bilgisayar ortamına nasıl aktarılacağına bakılır. Analizde ne olduğunu anladığınız işi nasıl çözümleyeceğinize, yazılımı hangi modüllerle oluşturacağınıza karar vermek ve bunu modellemek tasarımda yapılır. Tasarımdan sonra iş, programcılara tek tek aktarılır ve kodlama aşamasıyla devam eder.

10.3 Dört Temel Adım

Kodlanan programdan sonraki adım sorulduğunda sınıftan "test" cevabı geldi. Hoca temel adımları şöyle yazdı:

1. **Gereksinim analizi (requirements analysis):** "Requirement" kelimesinin karşılığıdır. Hoca, İTÜ'deki bazı hocaların "ister çözümleme" terimini kullandığını, kendilerinin ise başka üniversitelerdeki birçok hocayla birlikte "gereksinim analizi" terimini daha anlaşılır bulduğunu söyledi.
2. **Tasarım (design).**
3. **Gerçekleme (implementation):** Kodlama da denir, ama hocaya göre "gerçekleme" demek daha doğrudur, çünkü bu adımda yalnızca kod değil, veri tabanı da oluşturulur.
4. **Test.**

Bunlar yazılım geliştirme sürecinin temel adımlarıdır, yani yazılım geliştirirken asıl yapılan işlerdir.

11. Destek (Şemsiye) Aktiviteleri ve Yazılım Konfigürasyon Yönetimi

11.1 Destek Aktiviteleri

Temel adımları yaparken işin daha kaliteli olmasını isteriz. Bunun için temel adımlara destek olan aktiviteler vardır. Hoca bunları **yazılım geliştirme sürecinin destek aktiviteleri** olarak adlandırdı:

- **Proje planlama:** Kaynak planlama, zaman planlama, görev atama (kim hangi işi yapacak). Yalnızca planlamayla kalmaz, projenin takip edilmesini de içerir.
- **Kalite yönetimi:** Kaliteli yazılım geliştirmek için nelere önem verileceği; kalite planının yapılması, kalite ekibinin görevlendirilmesi, hangi standardın kullanılacağının belirlenmesi.
- **Yazılım ölçümü:** Hazırlanan yazılımın ölçülmesi ve başlangıçta ne büyüklükte bir yazılım istendiğinin belirlenmesi.
- **Risk analizi:** Geliştirme sürecinde hangi risklerin olduğu, bu risklerin neler olduğu ve nasıl takip edileceği. Riskler sürekli takip edilmelidir.
- **Yazılım konfigürasyon yönetimi:** Hocaya göre en önemli destek aktivitelerinden biri.

Bu aktiviteler temel adımların daha iyi gerçekleştirilmesini sağlar.

11.2 Yazılım Konfigürasyonu

Konfigürasyon yönetimi öğrencilerin daha önce görmediği bir konuydu. Hoca önce donanım konfigürasyonundan yola çıktı.

Soru: Donanım konfigürasyonu dediğimizde ne anlıyoruz? **Cevap (hoca):** Bir sistemin donanım elemanlarının neler olduğunu ve bunlar arasındaki ilişkileri.

Yazılım konfigürasyonu da benzer şekilde yazılımın elemanları ve aralarındaki ilişkilerdir. Hoca, yazılımın elemanlarını dersin başında gördüklerini hatırlatarak sınıfa yazılımın ne olduğunu ve nelerden oluştuğunu sordu.

Soru: Yazılım nedir, nelerden oluşur? **Cevap (bir öğrenci, hocanın aktarımıyla):** Program, artı basılı ya da basılı olmayan belgeler, artı veri tabanıyla ilgili tanımlamalar, artı test verileri, yani test case'ler.

> **Yazılım konfigürasyonu:** Yazılımı oluşturan bütün elemanlar ve bu elemanlar arasındaki ilişkiler yazılım konfigürasyonunu oluşturur. Bunların yönetilmesi **yazılım konfigürasyon yönetimidir**.
>
> **Ek bilgi (derste anlatılmadı):** Uygulamada yazılım konfigürasyon yönetimi; kaynak kodun, belgelerin ve test verilerinin sürümlerinin izlenmesini, değişiklik isteklerinin denetlenmesini ve hangi sürümün hangi belgeyle eşleştiğinin kayıt altında tutulmasını kapsar. Günümüzde Git gibi sürüm denetim sistemleri bu işin en görünür araçlarıdır.

Örnek Sorular

Bu derste ekran görüntüsü kullanılmadı. Aşağıdaki sorular hocanın derste sözlü olarak sorduğu sorulardır.

1. (derste sözlü, 00:15:14) Nesneye yönelik programlama dersinde gördüğünüz diyagramlar nelerdir?

> **Çözüm (derste verilen):** Hocanın kendi cevabına göre o derste use case yapısı üzerinde çok durulmuyor; ardışıl (sequence) diyagramlar ve sınıf (class) diyagramları çiziliyor. Bu derste bunların hepsi nesneye yönelik çözümleme ve tasarım için kullanılacak.

2. (derste sözlü, 01:13:43) Mühendis nedir? Mühendislik kavramındaki iki anahtar kelime nedir?

> **Çözüm (derste verilen):** Mühendis, bir problemle karşılaştığında ilk akla gelen çözümü hemen uygulamayan, önce tasarım yapan ve yaptığı işi ölçebilen kişidir. "Mü-

hendis” kelimesi “hendese” (ölçme bilimi) kökünden gelir. İki anahtar kelime **ölçme** ve **tasarımdır**.

3. (derste sözlü, 01:43:55) CASE aracı nedir?

Çözüm (derste verilen): CASE, Computer-Aided Software Engineering demektir. CAD/CAM’e benzer şekilde, yazılım geliştirme sürecindeki işleri kolaylaştıran, hızlandıran ve rutin işleri yapan yazılımlardır. Örnekler: UML çizim araçları, analiz ve tasarım diyagramları arasında tutarlılık kontrolü yapan araçlar, Violet, bütünleşik araç olarak Rational Rose.

4. (derste sözlü, 01:48:39) Yazılım geliştirme sürecinin temel adımları nelerdir?

Çözüm (derste verilen): Gereksinim analizi, tasarım, gerçekleme (kodlama ve veri tabanının oluşturulması) ve test.

5. (derste sözlü, 01:52:25) Analiz modeli ile tasarım modelinin farkı nedir?

Çözüm (derste verilen): Analiz “ne?” sorusuna cevap verir: problem nedir, müşteri ne istiyor, hangi raporlar ve hangi kalıcı veriler gerekiyor. Tasarım “nasıl?” sorusuna cevap verir: iş bilgisayar ortamına nasıl aktarılacak, yazılım hangi modüllerden oluşacak.

6. (derste sözlü, 02:06:12) Donanım konfigürasyonu dediğimizde ne anlıyoruz? Buradan yazılım konfigürasyonu nasıl tanımlanır?

Çözüm (derste verilen): Donanım konfigürasyonu, donanım elemanları ve aralarındaki ilişkilerdir. Yazılım konfigürasyonu da yazılımı oluşturan bütün elemanlar ve aralarındaki ilişkilerdir. Bunların yönetimi yazılım konfigürasyon yönetimidir.

7. (derste sözlü, 02:07:42) Yazılım nedir, nelerden oluşur?

Çözüm (derste verilen): Program, basılı ya da elektronik belgeler, veri tabanıyla ilgili tanımlamalar ve test verileri (test case’ler).

Dersin Toparlaması

Bu derste önce yazılımın yalnızca program olmadığı, program, belgeler, veri tanımları ve test verilerinin bütünü olduğu görüldü. Yazılım fiziksel olarak aşınmasa da zamanla yetersiz kalır. Bu yüzden bakım gerekir ve yazılım, fikrin doğmasından kullanım dışı bırakılmasına kadar bir yaşam döngüsü izler. Ardından yazılım türleri (sistem, bilimsel, gömülü, uygulama, COTS, ürün hattı, kurumsal), yeniden kullanılabilirlik ve eski yazılım kavramları ele alındı. Müşteri, yönetim ve programcı açısından yazılım hakkındaki yanılgılar, gereksinimlerin baştan doğru belirlenmesinin, planlamanın, teslim sonrası desteğin ve her aşamada kalite ölçümünün önemini gösterdi.

İkinci bölümde mühendisliğin iki anahtar kelimesi olan ölçme ve tasarımdan yola çıkılarak yazılım mühendisliği, “kaliteli yazılımı en ekonomik yolla elde etmek üzere mühendislik prensiplerinin kullanılması” ve IEEE’nin “yaşam döngüsüne sistematik, disiplinli ve ölçülebilir bir yaklaşımın uygulanması” tanımlarıyla açıklandı. CASE araçları tanıtıldı. Sürecin temel adımları (gereksinim analizi, tasarım, gerçekleme, test) ile destek aktiviteleri (proje planlama, kalite yönetimi, yazılım ölçümü, risk analizi, yazılım konfigürasyon yönetimi) ortaya kondu. Gelecek hafta süreç adımları biraz daha netleştirilecek ve yazılım geliştirme süreç modelleri, yani klasik modeller ve ardından çevik (agile) modeller işlenecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.